

O Caminho entre Palavras

Neste exercício, você deve escrever um programa que, dadas uma palavra de "início" e uma palavra de "fim" de mesmo comprimento, encontre um caminho mínimo de "início" para "fim" trocando apenas uma letra por vez. Por exemplo:

início: vela

fim: galo

caminho: vela -> gela -> gala -> galo

O caminho só pode possuir palavras que existam num arquivo de vocabulário dado, chamado “vocabulario.txt”, e que estará na mesma pasta que o arquivo .py do programa. O arquivo de vocabulário é um arquivo texto que contém uma palavra por linha. O seu programa deve ler esse arquivo e considerá-lo na descoberta do caminho.

Considere que todas as palavras fornecidas ao programa estarão sempre em letras minúsculas e sem acentos. Considere também que o programa sempre receberá um “início” e um “fim” que estão no vocabulário, sendo “início” diferente de “fim”.

O arquivo de vocabulário disponibilizado junto com o enunciado do EP contém palavras com até cinco letras. Você pode/deve testar o seu programa usando outros arquivos de vocabulário. Mas com palavras e vocabulários maiores, a execução do programa pode demorar bastante tempo.

Como Encontrar o Caminho Mínimo

As possibilidades de caminhos a partir de uma palavra podem ser representadas por meio de uma “árvore” (de ponta cabeça!), como a da Figura 1 abaixo, em que:

- O nó raiz é a palavra de início (ex. oca);
- Os nós são sempre palavras que existem no vocabulário;
- As palavras em todos os nós têm o mesmo comprimento;
- Um nó se ramifica em nós filhos, cada qual possuindo uma palavra que difere da de seu nó pai em apenas uma letra;
- Cada palavra aparece na árvore apenas uma vez;
- Os números nos nós indicam a ordem em que foram gerados (alterando uma letra, da esquerda para direita, em ordem alfabética).

Como primeiro exemplo, a Figura 1 mostra a árvore que deve ser gerada para encontrar o caminho entre a palavra de início “oca” e a palavra de fim “cla”.

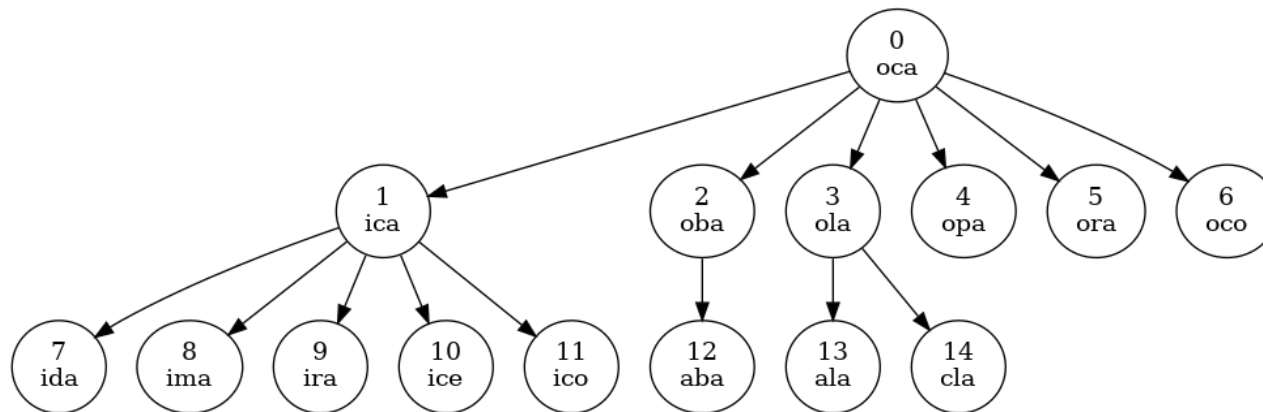


Figura 1: Árvore para início “oca” e fim “cla”.

Observe que mudando uma só letra de "oca", podemos obter: "aca", "bca", "cca", ..., "zca", "oaa", "oba", "oda", ..., "oza", "ocb", "occ", "ocd", ..., "ocz". Mas nem todas essas opções aparecem como nós filhos de "oca" na Figura 1 porque a árvore só pode conter palavras que existem no vocabulário.

Como segundo exemplo, a Figura 2 mostra a árvore que deve ser gerada para encontrar o caminho entre a palavra de início "foz" e a palavra de fim "giz".

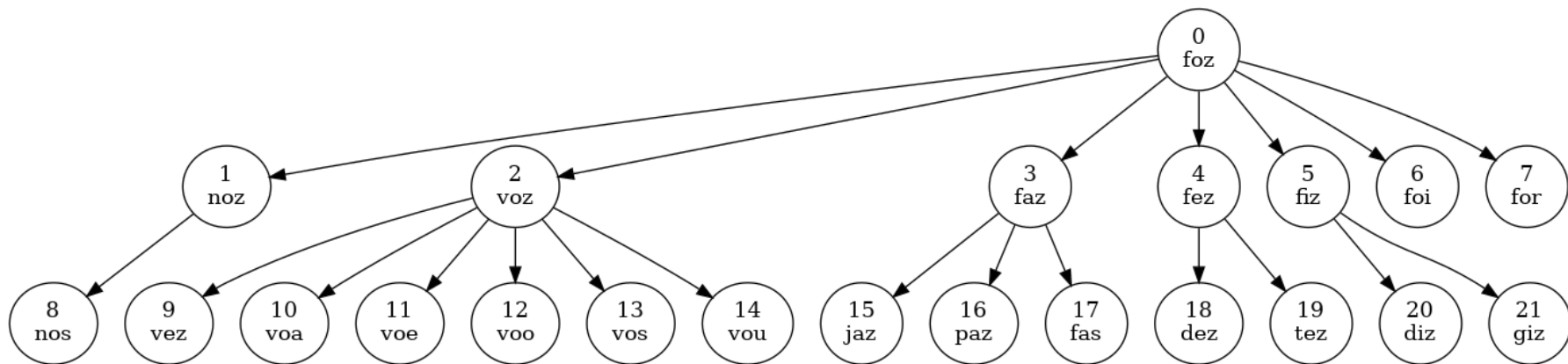


Figura 2: Árvore para início “foz” e fim “giz”.

O seu programa deve, a partir da palavra de início, gerar os nós da árvore na mesma ordem que a mostrada nos exemplos. Seu programa deve parar de gerar nós quando encontrar a palavra de fim (como nas Figuras 1 e 2) ou quando não houver mais novas palavras possíveis.

A árvore pode ser armazenada como uma lista de nós, onde cada nó é uma lista contendo uma palavra e a posição/índice do seu nó pai. Dessa forma, é possível “percorrer” o caminho entre um nó qualquer e o nó raiz.

Por exemplo, a lista correspondente à árvore da Figura 2 é a mostrada a seguir. Note que o “pai” do nó raiz é o -1 (ele não tem pai):

```
[ ['foz', -1], ['noz', 0], ['voz', 0], ['faz', 0], ['fez', 0], ['fiz', 0], ['foi', 0], ['for', 0],
  ['nos', 1], ['vez', 2], ['voa', 2], ['voe', 2], ['voo', 2], ['vos', 2], ['vou', 2], ['jaz', 3],
  ['paz', 3], ['fas', 3], ['dez', 4], ['tez', 4], ['diz', 5], ['giz', 5] ]
```

O Que Deve Ser Feito

Para a solução do problema, você deve implementar e usar as seguintes funções.

```
def obtemPalavrasProximas(palavra, vocabulario):
```

Esta função recebe dois parâmetros:

1. `palavra`: a palavra da qual se deseja encontrar as palavras próximas e
2. `vocabulario`: lista das palavras do vocabulário.

Esta função devolve uma lista de palavras que diferem de `palavra` em apenas uma letra. Nesta lista, primeiro aparecem as palavras que diferem na primeira letra mais à esquerda, depois as que diferem na segunda letra mais à esquerda, e assim por diante. A lista devolvida contém apenas palavras existentes na lista `vocabulario`.

```
def criaArvoreDeBusca(inicio, fim, vocabulario):
```

Esta função recebe três parâmetros:

1. `inicio`: palavra de início do caminho a ser buscado,
2. `fim`: palavra de fim do caminho a ser buscado e
3. `vocabulario`: lista de palavras do vocabulário

Esta função devolve uma lista com os nós da árvore de busca de caminho entre `inicio` e `fim`. Cada nó é uma lista contendo uma palavra e a posição/índice do seu nó pai. Portanto, a função devolve uma lista de listas. Os nós possuem apenas palavras existentes na lista de palavras `vocabulario`.

```
def obtemCaminho(inicio, fim, vocabulario):
```

Esta função recebe três parâmetros:

1. `inicio`: palavra de início do caminho a ser buscado,
2. `fim`: palavra de fim do caminho a ser buscado e
3. `vocabulario`: lista de palavras do vocabulário.

Esta função devolve uma lista com as palavras do caminho entre `inicio` e `fim`. O caminho contém apenas palavras existentes na lista `vocabulario` e também inclui `inicio` e `fim`. Caso não exista caminho de `inicio` à `fim`, a função devolve uma lista vazia.

Além dessas funções, você deve escrever na função `main()` um programa para oferecer três opções de execução:

Opção 1. Teste da função `obtemPalavrasProximas`. Nesta opção, seu programa deve ler do teclado uma palavra constante no arquivo `vocabulario.txt`. Esta palavra lida do teclado será o parâmetro `palavra` da função `obtemPalavrasProximas`. Em seguida, seu programa deve imprimir a `palavra` (parâmetro da função) e a respectiva lista de palavras próximas de `palavra` (lista de saída da função).

Opção 2. Teste da função `criaArvoreDeBusca`. Nesta opção, seu programa deve ler do teclado as palavras de início e fim, que serão os parâmetros `inicio` e `fim` da função `criaArvoreDeBusca`. Em seguida, seu programa deve imprimir o tamanho da árvore (lista de saída) e a árvore propriamente dita.

Opção 3. Teste da função `obtemCaminho`. Nesta opção, seu programa deve ler do teclado as palavras de início e fim, que serão os parâmetros `inicio` e `fim` da função `obtemCaminho`. Em seguida, seu programa deve imprimir o comprimento do caminho entre `inicio` e `fim` e o caminho propriamente dito, caso exista caminho. Caso contrário, seu programa deve imprimir que não existe caminho entre `inicio` e `fim`.

Seu programa deve permitir que o usuário possa executar essas três opções diversas vezes até que seja escolhida a opção zero.

Exemplos de Execução do Programa

Nos exemplos, tudo que aparece em **negrito** foi digitado pelo usuário. Os demais textos foram impressos pelo programa.

Exemplo 1 (Testa a função `obtemPalavrasProximas`)

```
Digite a opcao: 1
Digite uma palavra: berro
Palavras proximas de berro: ['cerro', 'ferro', 'serro', 'barro', 'borro', 'burro', 'beiro', 'berco',
'berra', 'berre']
Digite a opcao: 1
```

```
Digite uma palavra: flor
Palavras proximas de flor: []
Digite a opcao: 0
```

Exemplo 2 (Testa a função criaArvoreDeBusca)

```
Digite a opcao: 2
Digite a palavra de inicio: berro
Digite a palavra de fim: berco
Quantidade de nos da arvore: 9
Arvore: [['berro', -1], ['cerro', 0], ['ferro', 0], ['serro', 0], ['barro', 0], ['borro', 0], ['burro', 0], ['beiro', 0], ['berco', 0]]
Digite a opcao: 2
Digite a palavra de inicio: flor
Digite a palavra de fim: vaso
Quantidade de nos da arvore: 1
Arvore: [['flor', -1]]
Digite a opcao: 2
Digite a palavra de inicio: as
Digite a palavra de fim: kg
Quantidade de nos da arvore: 58
Arvore: [['as', -1], ['es', 0], ['os', 0], ['ah', 0], ['ai', 0], ['ao', 0], ['ar', 0], ['em', 1], ['eu', 1], ['ex', 1], ['oh', 2], ['oi', 2], ['ou', 2], ['ih', 3], ['uh', 3], ['li', 4], ['ri', 4], ['si', 4], ['ti', 4], ['ui', 4], ['vi', 4], ['xi', 4], ['do', 5], ['lo', 5], ['mo', 5], ['no', 5], ['po', 5], ['so', 5], ['ir', 6], ['im', 7], ['um', 7], ['nu', 8], ['tu', 8], ['ia', 13], ['ue', 14], ['la', 15], ['le', 15], ['ra', 16], ['sa', 17], ['se', 17], ['te', 18], ['tv', 18], ['va', 20], ['ve', 20], ['xa', 21], ['da', 22], ['de', 22], ['ma', 24], ['me', 24], ['na', 25], ['pa', 26], ['pe', 26], ['ca', 33], ['fa', 33], ['ha', 33], ['ja', 33], ['fe', 34], ['ze', 34]]
Digite a opcao: 0
```

Exemplo 3 (Testa a função `obtemCaminho`)

```
Digite a opcao: 3
Digite a palavra de inicio: berro
Digite a palavra de fim: berco
A distancia entre berro e berco é 1
berro -> berco
Digite a opcao: 3
Digite a palavra de inicio: foz
Digite a palavra de fim: giz
A distancia entre foz e giz é 2
foz -> fiz -> giz
Digite a opcao: 3
Digite a palavra de inicio: flor
Digite a palavra de fim: vaso
Nao existe caminho entre flor e vaso
Digite a opcao: 3
Digite a palavra de inicio: as
Digite a palavra de fim: kg
Nao existe caminho entre as e kg
Digite a opcao: 0
```

Exemplo 4 (Testa as três funções)

```
Digite a opcao: 1
Digite uma palavra: clube
Palavras proximas de clube: ['coube']
```

```
Digite a opcao: 2
Digite a palavra de inicio: clube
Digite a palavra de fim: roupa
Quantidade de nos da arvore: 16
Arvore: [['clube', -1], ['coube', 0], ['roube', 1], ['soube', 1], ['coibe', 1], ['couve', 1], ['rouba',
2], ['roubo', 2], ['coice', 4], ['coiba', 4], ['coibi', 4], ['coibo', 4], ['houve', 5], ['louve', 5],
['rouca', 6], ['roupa', 6]]
Digite a opcao: 3
Digite a palavra de inicio: clube
Digite a palavra de fim: roupa
A distancia entre clube e roupa é 4
clube -> coube -> roube -> rouba -> roupa
Digite a opcao: 0
```

Atenção: Imprima os textos informativos ao usuário exatamente da mesma forma que apresentado nos exemplos de execução acima. Ao ser executado com os mesmos dados de entrada dos exemplos acima, o seu programa deverá exibir mensagens **exatamente iguais** às dos exemplos, inclusive os espaços.

As árvores de palavras correspondentes aos exemplos de execução do programa são mostradas a seguir.

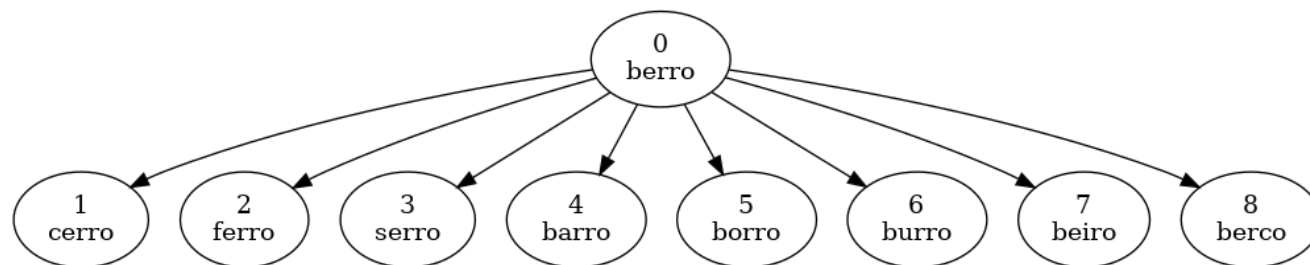


Figura 3: Árvore para início “berro” e fim “berco”.

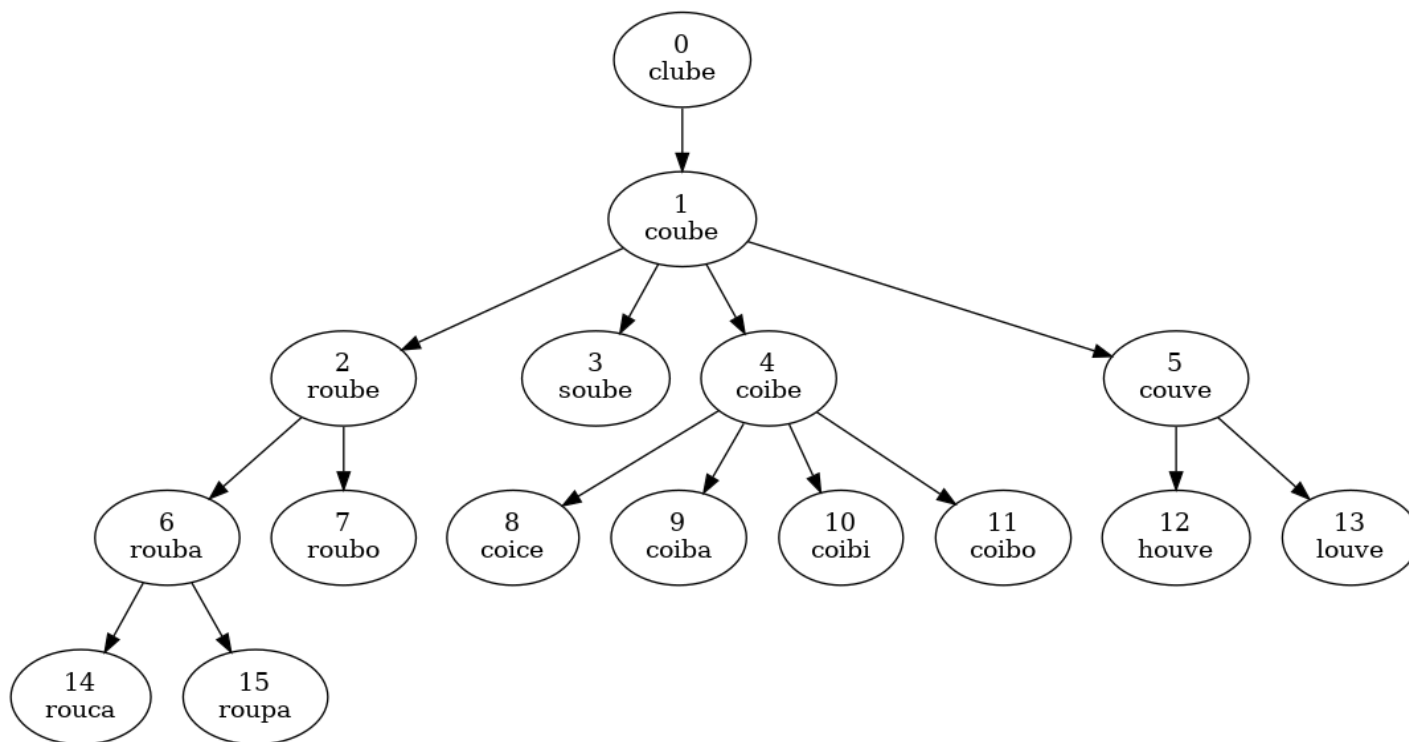


Figura 4: Árvore para início “clube” e fim “roupa”.

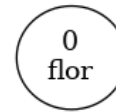


Figura 5: Árvore para início “flor” e fim “vaso” (não há caminho entre essas palavras).

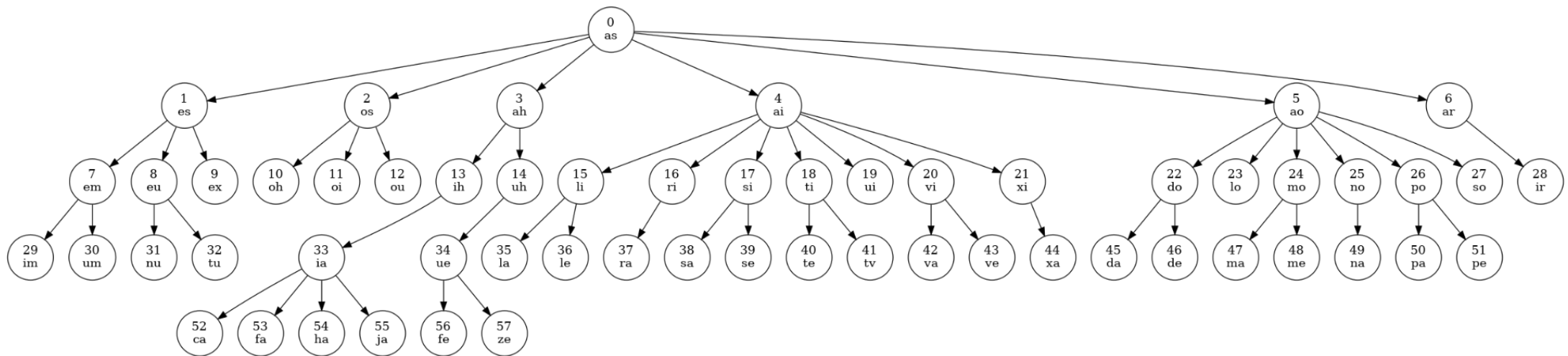


Figura 6: Árvore para início “as” e fim “kg” (não há caminho entre essas palavras).